

数据结构期中极限备考

刘青乐 2025.11.08

Chapter 1: Algorithm

TM and RAM

- TM (example: Increase, Decrease)
- RAM (example: plus, minus, multiply, divide(floor & ceiling))

级数

$$1^d + 2^d + \dots + n^d = O(n^{d+1})$$

$$a^0 + a^1 + \dots + a^n = O(a^n)$$

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = O(\log n)$$

$$\ln 1 + \ln 2 + \ln 3 + \dots + \ln n = O(n \log n)$$

$$1 \ln 1 + 2 \ln 2 + 3 \ln 3 + \dots + n \ln n = O(n^2 \log n)$$

$$1 \cdot 2^1 + 2 \cdot 2^2 + 3 \cdot 2^3 + \dots + n \cdot 2^n = O(n \cdot 2^n)$$

Master and Akra-Bazzi

$$T(n) = aT(n/b) + O(g(n))$$

取 $n^{\log_b a}$ 和 $g(n)$ 中的高阶项即为 $T(n)$, 若同阶即 $g(n) = \Theta(n^{\log_b a} \cdot \log^k n)$, 则 $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

$$T(n) = \sum_{i=1}^k a_i \cdot T(b_i \cdot n + h_i(n)) + O(g(n))$$

取 p 使得 $\sum_{i=1}^k a_i \cdot b_i^p = 1$, 则

$$T(n) = \Theta(n^p \cdot (1 + \int_1^n \frac{g(u)}{u^{p+1}} du))$$

Chapter 2: Vector

三种二分查找

```
// Version A
int binSearch1(int* vec, const int& e, int low, int high){
    while (low < high){
        int mid = (low + high) / 2;
        if (e < vec[mid])
            high = mid;
        else if (vec[mid] < e)
            low = mid + 1;
        else
            return mid;
    } // exit when low == high
    return -1; // fail
}
```

重要等式 (Version A):

$$(ASL_{Succ} + 1) \cdot n = (n + 1) \cdot ASL_{Fail}$$

```
// Version B
int binSearch2(int* vec, const int& e, int low, int high){
    if (e < vec[low])
        return low - 1; // fail (special judge only in versionB)
    while (low + 1 < high){
        int mid = (low + high) / 2;
        if (e < vec[mid])
            high = mid;
        else
            low = mid;
    } // exit when low + 1 == high
    return low;
}
```

约定 (Version B): 返回不大于e的最大下标

```
// Version C
int binSearch3(int* vec, const int& e, int low, int high){
    while (low < high){
        int mid = (low + high) / 2;
        if (e < vec[mid])
            high = mid;
        else
            low = mid + 1;
    } // exit when low == high
    return low - 1;
}
```

约定 (Version C): low-1是不大于e的最大下标, high是大于e的最小下标

Bitmap

- $M[k \gg 3] \& (0x80 \gg (k \& 0x07))$
- $T[F(k)] == k, F[T(k)] == k$
- Hopcraft: clear $F[T[top]] = F[k], T[F[k]] = T[top]$

Chapter 3: List

四种排序

容器 / 算法	最好	最坏	平均	就地	在线	输入敏感
vector 冒泡	O(n)	O(n²)	O(n²)	✓	✗	✓
vector 选择	O(n²)	O(n²)	O(n²)	✓	✗	✗
vector 插入	O(n)	O(n²)	O(n²)	✓	✓	✓
vector 归并	O(n log n)	O(n log n)	O(n log n)	✗	✗	✗
list 冒泡	O(n)	O(n²)	O(n²)	✓	✗	✓
list 选择	O(n²)	O(n²)	O(n²)	✓	✗	✗
list 插入	O(n)	O(n²)	O(n²)	✓	✓	✓
list 归并	O(n log n)	O(n log n)	O(n log n)	✓	✗	✗

Chapter 4: Stack and Queue

栈混洗

$$SP(n) = \sum_{k=1}^n SP(k-1) \cdot SP(n-k) = catalan(n) = \frac{(2n)!}{(n+1)!n!}$$

表达式求值与RPN

```
const char pri[9][9] = { //优先级快查表[栈顶][当前]
/*      |----- 当前运算符 -----| */
/*      +   -   *   /   ^   !   (   )   \0 */
/*--  + */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
/* | - */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
/* 栈 * */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 顶 / */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
/* 运 ^ */ '>', '>', '>', '>', '>', '<', '<', '>', '>',
/* 算 ! */ '>', '>', '>', '>', '>', '>', ' ', '>', '>',
/* 符 ( */ '<', '<', '<', '<', '<', '<', '<', '~', ' ',
/* | ) */ ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
/*-- \0 */ '<', '<', '<', '<', '<', '<', '<', ' ', '~' };
```

非法情况：

- prio['!']['(']
- prio['(']['\0']
- prio['\0'][')']
- prio[')']['Any']

```

// 表达式求值（同RPN）
double evaluate( char* S, char* RPN ) { //对表达式S求值，并转换为RPN
    Stack<double> opnd; Stack<char> optr; //运算数栈、运算符栈
    for ( optr.push('\0'); !optr.empty(); ) { //哨兵'\0'入栈，然后顺序扫描
        if ( isdigit( *S ) ) //遇到操作数，则读入，并续接至RPN
            readNumber( S, opnd );
        append( RPN, opnd.top() );
    }
    else //若遇到运算符，则视其与栈顶运算符之相对优先级
        switch( prio( optr.top(), *S ) ) { //分别处理
            case '<': //栈顶运算符优先级更低时
                optr.push( *S ); S++; //计算推迟，当前运算符进栈
                break;
            case '>': //反之
                append( RPN, optr.top() ); //栈顶运算符续接至RPN
                opnd.push( calcu( opnd, optr.pop() ) ); //实施计算，结果入栈（calcu()自行实现）
                break;
            case '~': //括号或哨兵'\0'
                optr.pop(); S++; //脱括号并转至下一字符
                break;
        } //switch
    return opnd.pop(); //唯一尚存的操作数，即为计算结果
}

```

Chapter 5: Tree

遍历

```

// preorder
void preorder (TreeNode* x) {
    stack<TreeNode*> S;
    S.push(x);
    while (!S.empty()) {
        x = S.top();
        S.pop();
        while (x) {
            visit(x);
            S.push(x->right);
            x = x->left;
        }
    }
}

```

```

// inorder
void inorder (TreeNode* x) {
    stack<TreeNode*> S;
    S.push(x);
    while ( x || !S.empty()) {
        while (x) {
            S.push(x);
            x = x->left;
        }
        x = S.top();
        S.pop();
        visit(x);
        x = x->right;
    }
}

// postorder
void postorder (TreeNode* x) {
    stack<TreeNode*> S;
    S.push(x);
    while (!S.empty()) {
        if (S.top() != x->parent) { // not parent (but right brother), find LeftMostLeaf
            x = S.top();
            while(x){
                if(x->right)
                    S.push(x->right);
                if(x->left){
                    S.push(x->left);
                    x = x->left;
                }
                else
                    x = x->right;
            }
        }
        x = S.top();
        S.pop();
        visit(x);
    }
}

//levelorder, 略

```

辅助栈规模（自创）：

先序遍历：松界： $h(T)$ ；紧界： $O(\max(la(v)|v \in T))$ ；更准确地说：最长的向左延伸藤的长度

中序遍历：松界： $h(T)$ ；紧界： $O(\max(la(v)|v \in T))$ ；更准确地说：最长的向左延伸藤的长度

后序遍历：松界： $2h(T)$ ；紧界： $O(2 \max(la(v)|v \in T))$ ；更准确地说：最大la数+最长的向左延伸藤的长度

层次遍历： $\lceil \frac{n}{2} \rceil$